

Spiffy: Multiplying Diffusion LLM Acceleration via Lossless Speculative Decoding

Sudhanshu Agrawal¹

Mingu Lee^{*}

Risheek Garrepalli

Christopher Lott^{*}

Raghav Goel

Fatih Porikli^{*}

Qualcomm AI Research[†]

{sudhagra, rgarrepa, raghgoel, mingul, clott, fporikli}@qti.qualcomm.com

Abstract

Diffusion LLMs (dLLMs) have recently emerged as a powerful alternative to autoregressive LLMs (AR-LLMs) with the potential to operate at significantly higher token generation rates. However, currently available open-source dLLMs often generate at much lower rates, typically decoding only a single token at every denoising timestep in order to maximize output quality. We present Spiffy, a speculative decoding algorithm that accelerates dLLM inference by **2.8–3.1×** while provably preserving the model’s output distribution. This work addresses the unique challenges involved in applying ideas from speculative decoding of AR-LLMs to the dLLM setting. Spiffy proposes draft states by leveraging the dLLM’s distribution itself in an auto-speculative manner. This approach is efficient and effective, and eliminates the overheads of training and running an independent draft model. To structure the candidate draft states, we propose a novel directed draft graph which is uniquely designed to take advantage of the bidirectional, block-wise nature of dLLM generation and can be verified in parallel by the dLLM. To further optimize the structure of these draft graphs, we introduce an efficient, offline calibration algorithm that procedurally determines high-quality graph configurations. These optimized draft graphs, enabling increased acceptance rates, lead to a significant boost in the overall speedup achieved by the system. Crucially, Spiffy is also complementary to other recent innovations in improving dLLM generation speeds such as KV-caching and multi-token unmasking. We demonstrate that when combined with such parallel decoding algorithms, Spiffy is able to effectively multiply the benefits of these methods leading to total speedups of up to **7.9×**.

1 Introduction

A majority of large language models (LLMs) today are autoregressive LLMs (AR-LLMs) (Brown et al. 2020). These models operate in a left-to-right fashion, generating new tokens that simply depend on the previous tokens in the sequence. Such LLMs have demonstrated impressive capabilities in a variety of domains. Yet, due to their autoregressive nature, they are constrained to generating only a single token per model inference. As overall latency is typically

dominated by the total model inference time, this constraint slows down the LLM application as a whole.

More recently, the emergence of diffusion LLMs (dLLMs) (Nie et al. 2025; Inception AI Labs et al. 2025) represents a paradigm shift in language modeling. Unlike AR-LLMs, dLLMs are inherently parallel in nature and are not bound by a strict causal factorization. Instead, they model the joint distribution of over entire token blocks, leveraging bidirectional attention (Devlin et al. 2019), and allow tokens to be decoded in arbitrary orders. dLLMs frame token decoding as an iterative denoising process, taking a sequence from an initial high-noise state to its final low-noise state, representing the generated token sequence. While commercial dLLMs such as Mercury (Inception AI Labs et al. 2025) and Gemini Diffusion (Google 2025) may achieve token rates in excess of 1000 tokens/second, open-sourced dLLMs such as LLaDA (Nie et al. 2025) and Dream (Ye et al. 2025) do not yet achieve these decoding speeds. In particular, to maintain output quality, these dLLMs generate only a *single token* per model inference, restricting their efficiency.

On the other hand, recent work on AR-LLMs attempts to circumvent their autoregressive constraints via speculative decoding (Leviathan et al. 2023; Chen et al. 2023). This has emerged as a popular and powerful method to accelerate AR-LLMs due to its accuracy guarantee and potential for efficient deployment. However, transferring the ideas developed for AR-LLM speculation to dLLMs encounters fundamental challenges. Since dLLM token generations have bidirectional dependence and model the joint distribution over sequences, drafting and verification must take this into account to remain lossless. Typical speculative strategies, such as tree-based drafting, also require substantial re-thinking. Importantly, the choice of a compatible and efficient drafter is essential to enable speculation in dLLMs. In this work, we address these challenges and introduce speculative decoding acceleration into dLLM systems.

Spiffy (Speculation for Diffusion LLM Efficiency), is a novel algorithm that accelerates dLLM inference with a *draft then verify* strategy which provably maintains output quality. During generation, Spiffy samples draft states from the dLLM distribution itself, avoiding the need for, and costs of an auxiliary draft model. Spiffy proposes these drafts in the form of a directed draft graph to maximize their acceptance. While these draft graphs are similar in motivation to

¹Corresponding author

^{*}Senior authors listed alphabetically by last name.

[†]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

draft trees (Li et al. 2024; Miao et al. 2024; Jeon et al. 2024), Spiffy’s draft graphs respect and also take advantage of the bidirectional nature of dLLM generation. In contrast to trees, nodes in these draft graphs may have multiple parent nodes and consequently multiple pathways to being accepted. This drafting structure more effectively captures the state dependencies inherent to dLLM generation.

To further enhance these draft graphs and maximize their efficacy, we additionally introduce a novel calibration procedure to determine optimized draft graph structures. This calibration procedure is performed once, offline, and the resulting graph configurations remain fixed throughout the denoising process, significantly increasing acceptance rates during generation.

Spiffy is versatile and extensible as a framework, remaining compatible with alternate drafting mechanisms and arbitrary denoising rates. This allows for it to be combined with other parallel decoding techniques for dLLMs and optimizations such as KV-caching to further boost inference speeds. In particular, we are able to effectively combine Spiffy with the parallel decoding scheme proposed by *fast-dllm* (Wu et al. 2025) to multiply its speedup benefits.

Thus, we propose Spiffy as a powerful, plug-in acceleration solution for a variety of dLLM systems. We summarize our key contributions below.

1. **Drafting and verification for dLLMs** We introduce Spiffy, a novel, versatile framework for speculative decoding of block-wise dLLMs. We propose seamless multi-token drafting and define verification as a lossless procedure, proven formally and experimentally.
2. **Auto-speculation** We demonstrate the efficacy of this framework with an auto-speculative approach, leveraging the current dLLM distribution to speculate draft states, thus avoiding the need to train and run a draft model.
3. **Directed draft graphs** We define directed draft graphs, a generalization of speculative draft trees, designed to take advantage of the bidirectional nature of dLLMs and increase the likelihood of successive draft acceptances.
4. **Offline graph structure calibration** To further optimize these draft graph structures, we develop a novel calibration algorithm that procedurally determines high-quality graph configurations, boosting acceptance rates during speculation.

Our contributions are validated with rigorous experimentation and ablations with multiple open-source dLLMs (LLaDA-Base-8B, LLaDA-Instruct-8B, LLaDA-1.5-8B) with standard benchmarks (GSM8K, HumanEval, MATH, MBPP), enabling up to $7.9\times$ total speedup.

2 Related Work

Diffusion LLMs Diffusion models (Sohl-Dickstein et al. 2015; Ho et al. 2020) have become a dominant approach in generative modeling, particularly excelling in continuous domains such as image synthesis (Song et al. 2020; Saharia et al. 2022; Esser et al. 2024; Black Forest Labs et al. 2025). Their capacity for parallel generation has motivated their adaptation to language modeling, a discrete domain with

inherently different challenges. Early works explored structured denoising and discrete diffusion processes for text generation (Austin et al. 2021a; Lou et al. 2023), while more recent efforts have focused on token-level masking-based forward processes (Sahoo et al. 2024; Shi et al. 2024) and block-wise decoding (Arriola et al. 2025). Current open-source models implementing these ideas at scale include LLaDA (Nie et al. 2025) and Dream (Ye et al. 2025).

Accelerating Diffusion LLMs While dLLMs have the potential for parallel generation, in practice, many still decode one token at a time, thus requiring a large number of denoising steps and resulting in slow inference. To overcome this, recent work has explored simultaneous token updates based on notions of confidence determined intrinsically (Wu et al. 2025) or by an auxiliary AR-LLM (Israel et al. 2025), as well as the use of modified solvers to accelerate the reverse diffusion process (Shaul et al. 2024; Li et al. 2025; Luxembourg et al. 2025). A concurrent work by (Hong et al. 2025) proposes drafting and verification for dLLMs via shadow draft blocks. Additionally, efforts have been made to integrate key-value (KV) caching mechanisms to reduce redundant computation (Ma et al. 2025; Liu et al. 2025).

Speculative Decoding of AR-LLMs Speculative decoding (Leviathan et al. 2023; Chen et al. 2023) accelerates the inference of a large, target model with an efficient drafter. Generations from the drafter have seen several forms, most often tree-based (Miao et al. 2024; Jeon et al. 2024; Li et al. 2024). Other works reduce the overhead of the drafter (Lin et al. 2025) or have tried to do away with it entirely (Fu et al. 2024a; Zhang et al. 2023; Fu et al. 2024b). Recent methods have proposed improvements to the draft generation quality via training improvements (Goel et al. 2024), by allowing the drafter to leverage the final hidden features of the target model (Li et al. 2024), and by adding additional linear modules to predict multiple draft tokens at once (Cai et al. 2024). Recent works also explore the usage of a small diffusion language model as a draft model for a target AR-LLM (Christopher et al. 2024).

Speculation for Image Diffusion Models (Wang et al. 2024) and (De Bortoli et al. 2025) concurrently propose modifications to rejection sampling in the case of continuous-valued diffusion models and achieve impressive speedups in image generation latency. While (Wang et al. 2024) use an independent diffusion model as a draft model, (De Bortoli et al. 2025) and additionally (Hu et al. 2025), use the target model to speculate future states. A line of work additionally explores the usage of multi-sample generation and gradient-based MCMC within energy-based formulations (Du et al. 2020; Singhal et al. 2025).

3 Preliminaries

Masked Diffusion Language Models Masked diffusion language models, at inference time, define a reverse denoising process where a token sequence $X(T)$ is gradually unmasked to a valid sequence $X(0)$. Practically, this is often

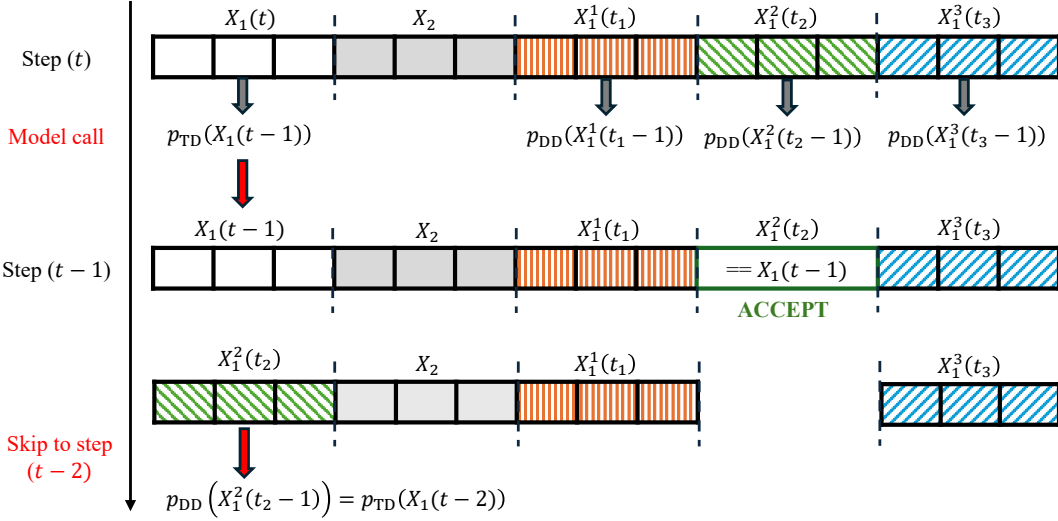


Figure 1: Spiffy’s lossless verification allows $X_1(t)$ to advance to state $X_1(t-2)$ with a single model inference. Draft blocks X_1^1, X_1^2, X_1^3 are appended to the end of the sequence and model inference is performed in parallel. Each draft is compared against $X_1(t-1)$ to check for acceptance. Since we have access to any accepted draft’s distribution at time $t-2$, we skip ahead and predict $X_1(t-2)$. We then repeat this process with the remaining drafts for further acceleration.

implemented by setting a generation length W and initializing the sequence with a specially designated MASK token as $X(T) = [\text{MASK}, \dots, \text{MASK}]$. This sequence is divided into N blocks of size L each and we denote the i^{th} block of X by X_i . The dLLM then fully unmask each block one by one, taking it from $X_i(T) \rightarrow X_i(0)$. In particular, at each timestep $t \in \{T, \dots, 0\}$, the dLLM is called to model:

$$p_\theta(X_i(t-1) | X(t)) \quad (1)$$

By sampling $X_i(t-1)$ from (1), we obtain the next state of the sequence where S_{t-1} additional tokens have been unmasked. Here, S_t defines the incremental number of tokens that need to be denoised to go from timestep $t+1 \rightarrow t$. S_t thus represents the rate at which a block is denoised. Open-source dLLMs such as LLaDA (Nie et al. 2025) and Dream (Ye et al. 2025) often set this parameter as $S_t = S = 1$ for all timesteps t , consequently unmasking 1 token per iteration. This setting allows these dLLM to achieve accuracies comparable to current AR-LLMs while setting $S > 1$ leads to a significant loss in model performance (Wu et al. 2025).

Latency in LLM systems is dominated by the model inference time and typically multiplies with the number of model forward calls. Thus, if S_t is set to 1 in order to maximize accuracy, the model must be called a total of W times in order to generate all W tokens, making such a setup expensive. At the same time, this scenario brings to mind proven solutions from the AR-LLM acceleration literature.

Speculative Decoding Speculative decoding (Leviathan et al. 2023; Chen et al. 2023) is a powerful technique designed to parallelize the decoding process of AR-LLMs while provably preserving their output distribution. A typical speculative decoding setup consists of a large *target* model that one wishes to accelerate and a second,

more efficient *drafter* that approximates it. This drafter may take many forms, as discussed in Sec. 2, and proposes a set of draft completions for the target model to verify in parallel. Concretely, given a prompt $X_P = [x_1, \dots, x_t]$, the target model has a target distribution p_{TD} and autoregressively models $p_{TD}(x_i | X_P, x_{t+1}, \dots, x_{i-1})$ for $i \in \{t+1, \dots, t+L\}$, thus generating L new tokens, $[x_{t+1}, \dots, x_{t+L}]$. This process is slow, so to accelerate it, the drafter models a draft distribution $p_{DD}(x_i | x_{<i})$ generating draft tokens $[\hat{x}_{t+1}, \dots, \hat{x}_{t+D}]$ for some draft length D . These draft tokens may then be *verified* in parallel by obtaining $p_{TD}(\cdot | x_{<i}, \hat{x}_{<i})$ and performing rejection sampling with the candidate draft tokens. In a greedy decoding setting, we may simply sample $x_i \sim p_{TD}(\cdot | x_{<i}, \hat{x}_{<i})$ and accept $\hat{x}_i$ if it equals x_i . If we accept M of these draft tokens, we have effectively sampled $[x_{t+1}, \dots, x_{t+M}]$ using only a single target model inference.

4 Method

Spiffy casts the denoising process of dLLMs in a speculative decoding framework. Rather than speculating individual tokens, Spiffy speculates the state of the sequence over denoising timesteps, allowing us to ‘skip ahead’ in the unmasking process while maintaining output quality.

Notation Suppose in the setting defined in Sec. 3, we are currently denoising block $k \in \{1, \dots, N\}$ and X_k is observed at a denoising timestep $t \in [T, 0]$ as $X_k(t)$. Since the rest of the sequence is unchanged while X_k is being unmasked, let us refer to the remaining blocks of the sequence collectively as X' . Thus the token sequence as a whole is $X(t) = X'; X_k(t)$ (the sequence with $X_k(t)$ taking the position of the k^{th} block).

4.1 Draft Blocks

When unmasking any given token, it is essential to consider the influence of every other token in the sequence. This is a characteristic property of bidirectional attention (Devlin et al. 2019) and to satisfy this condition, we consider drafts at the block level. We denote a **draft block** for X_k as $\hat{X}_k^m(t_m)$ with $m \in \{1, \dots, D\}$ for a number of drafts D . Each draft block represents a speculation of the state of the block at a timestep t_m and we refer to the set of current draft blocks as $\{\hat{X}_k^m\}$.

4.2 Verification

At timestep t , the current state of the target (true) sequence is X' ; $X_k(t)$. We use the dLLM to model two separate distributions, the target distribution at the next timestep:

$$p_{TD}(t-1) = p_\theta(X_k(t-1) | X'; X_k(t)) \quad (2)$$

and the draft distributions at their corresponding next timesteps:

$$p_{DD}^m(t_m-1) = p_\theta(X_k(t_m-1) | X'; \hat{X}_k^m) \quad (3)$$

This can be achieved in parallel, using a single model call with a custom attention mask and appropriate positional embeddings, not dissimilar to the strategies adopted by (Miao et al. 2024; Jeon et al. 2024; Li et al. 2024). We include an example of such a mask in Fig. 2.

To advance the sequence to the next timestep, S_{t-1} tokens are sampled from the target distribution and are unmasked. We then compare the state of the block to our draft candidates and accept any draft that matches. Since we already have access to the draft distributions of the accepted block, we may skip ahead in the denoising, and repeat the procedure with the remaining drafts. This verification method is described in detail in Algorithm 1 and visualized in Fig. 1.

Thus, if in the process of unmasking a block at a rate of S_t tokens, we accept drafts for timesteps $T_1, \dots, T_M$, we decrease the number of model calls required by a factor of,

$$\frac{\sum_t S_t}{\sum_t S_t - \sum_{i=1}^M S_{T_i}} \quad (4)$$

For example, if $S_t = 1$ for all timesteps and we accept M drafts, we achieve a total speedup of,

$$\frac{W}{W - M} \quad (5)$$

Since the overheads of Spiffy are negligible compared to the model inference time (Appendix B), this value will be close to the true speedup achieved by the algorithm. Moreover, we show that Algorithm 1 is *lossless*, that is, it preserves the output distribution of the model. We include a proof of this statement in Appendix A.1.

While Algorithm 1 is lossless in general, in certain settings, producing the exact draft probabilities $\{p_{DD}^m\}$ is computationally expensive. In particular, for full-sequence bidirectional decoding, achieving exact decoding requires construction of a batched attention mask to ensure that all the keys and values attended to by the draft blocks, in turn,

Algorithm 1: Spiffy Draft Block Verification

Given: unmasking rate $\{S_t\}$, *vocabulary* of size V
Input: current t , current $X_k(t)$, target p_{TD} , draft $\{p_{DD}^m\}$, drafts $\{\hat{X}_k^m\}$
(1) Extract marginals p_n for each token position
 $p_n \leftarrow p_{TD}(t-1)[n] \in [0, 1]^V$
(2) top-1 probability for each token position n
 $p_n^1, j_n \leftarrow \max, \arg \max_{v \in V} p_n(v) \forall n \in [1, L]$
(3) Unmask top- S_{t-1} positions
 $n_1, \dots, n_{S_{t-1}} \leftarrow \text{topKIndex}(S_{t-1}, [p_1^1, \dots, p_L^1])$
 $c_{n_i} \leftarrow \text{vocabulary}[j_{n_i}] \quad \forall i \in 1, \dots, S_{t-1}$
(4) Update the block with the new tokens
 $X_k(t-1) \leftarrow X_k(t)$
 $X_k(t-1)[n_i] \leftarrow c_{n_i} \quad \forall i \in 1, \dots, S_{t-1}$
 $t \leftarrow t-1$
(5) Draft verification loop
for $m \in \{1, \dots, D\}$ **do**
 if $t_m \neq t$ **then**
 continue
 end if
 if $\hat{X}_k^m == X_k(t)$ **then**
 # Accept
 $p_{TD}(t-1) \leftarrow p_{DD}^m(t-1)$
 Pop $\hat{X}_k^m$ from the draft list
 # Restart the algorithm from this state
 Verify($t, X_k(t), p_{TD}, \{p_{DD}^m\}, \{\hat{X}_k^m\}$)
 end if
end for
Output: $t, X_k(t)$

attend to the draft blocks. This would increase the computational requirements of the system by a factor proportional to the number of draft blocks and is hence infeasible. Instead, we choose to create a simple blockwise tree-attention mask structure that we see achieves near-lossless performance for full-sequence bidirectional dLLMs such as LLaDA in its default inference setting. Moreover, using this simple mask, we can achieve exact lossless speculative decoding when utilizing dual caches (Wu et al. 2025) or when decoding blockwise causal dLLMs such as SDAR (Cheng et al. 2025). In this work, we demonstrate near-lossless decoding on LLaDA models, which provides an upper bound on the error of our method, through numerical experiments in Appendix A.2.

4.3 Drafting

Draft Graphs We now desire draft blocks $\{\hat{X}_k^m\}$ for $m \in \{1, \dots, D\}$ such that their *acceptance is maximized*. In particular, we would like the opportunity to have *multiple* drafts accepted per model call. To this end, we structure the draft blocks to have a directed, *parent-child* relationship with each other and to the current block $X_k(t)$. Block A at timestep t_a

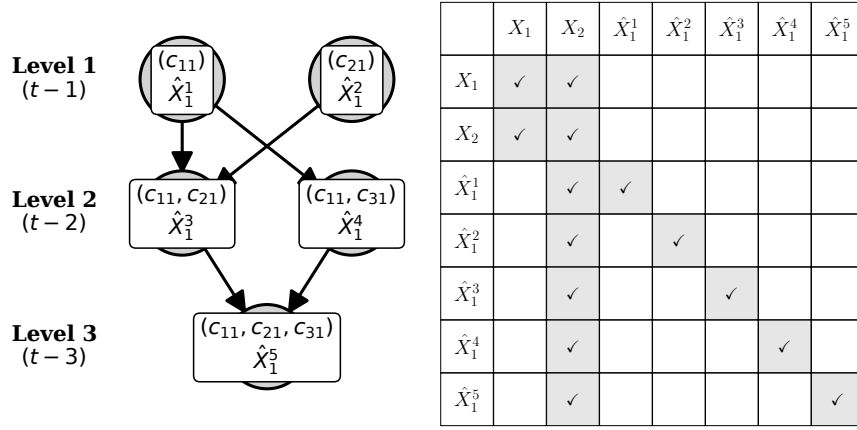


Figure 2: Directed draft graph where each node represents a draft block. Each draft block may have multiple parent states. In the first level, only a single token has been unmasked, c_{11} or c_{21} . In the second level, representing the second speculation timestep, c_{11} and c_{21} have been unmasked. This state can be reached if either c_{11} or c_{21} had been unmasked in the previous step, allowing multiple chances to reach the second draft level. The second level additionally contains c_{11}, c_{31} , which can be accessed through c_{11} even though c_{31} had not been drafted. In total, there are 3 routes through the graph that reach the third level c_{11}, c_{21}, c_{31} . These drafts may all be verified in parallel using a block-attention mask as shown to the right.

is defined to be a **parent** of block B at timestep t_b if:

$$t_b = t_a - 1 \quad (6)$$

$$|\text{unmasked}(A)| + S_{t_b} = |\text{unmasked}(B)| \quad (7)$$

$$\text{unmasked}(A) \subset \text{unmasked}(B) \quad (8)$$

Simply, block B should be a potential *next step* in the denoising process starting from block A . This property allows us to accept multiple drafts per model call since the acceptance of draft A means that we may potentially accept any children of A that were also drafted. In this manner, we may skip ahead multiple timesteps with a single model call.

We define $\text{children}(A)$ as the set of all possible blocks B that satisfy (6, 7, 8). We first choose a set of draft blocks, $\{\hat{X}_k^m\}(\text{level } 1) \subset \text{children}(X_k(t))$. These selections represent the *first level* of drafts. We then choose draft blocks for subsequent levels in a similar manner:

$$\{\hat{X}_k^m\}(\text{level } i) \subset \bigcup_{A \in \{\hat{X}_k^m\}(\text{level } i-1)} \text{children}(A) \quad (9)$$

Together, our draft blocks $\{\hat{X}_k^m\}$ take on the form of a **directed draft graph**. Notably, unlike draft trees used for speculative decoding of AR-LLMs, each draft block may have *multiple parents*, and thus *multiple pathways to being accepted*, a unique advantage of bidirectional attention. A visualization of such a draft graph is provided in Fig. 2.

Drafting Source Given this desired drafting structure, we now need a process by which the ‘contents’ (unmasked tokens) of a draft block may be determined. At any timestep, we have access to $p_{\text{TD}} = p_{\theta}(X_k(t-1) | X'; \hat{X}_k(t))$. We show in this section that this distribution may be leveraged to choose draft blocks. For brevity, ArgSort is assumed to be in descending order in the following relations.

First, with the marginals $p_1, \dots, p_L$ of the current distribution, we obtain an ordering of token positions:

$$n_1, \dots, n_L \leftarrow \text{ArgSort}(\max(p_1), \dots, \max(p_L)) \quad (10)$$

For each of these sorted positions n_i , we may determine the top- k token choices from the vocabulary of size V :

$$c_{i1}, \dots, c_{ik} \leftarrow \text{ArgSort}(p_{n_i}(v_1), \dots, p_{n_i}(v_V))[:k] \quad (11)$$

Thus, when considering candidates for unmasking, the ‘ranking’ of a candidate depends on both its position in the token sequence and its likelihood among token choices for that position. We can characterize this ranking by defining two new terms. For any token position $n \in \{1, \dots, L\}$ that we are considering unmasking, we define its **token position rank** and denote it by i

$$i := \text{ArgSort}(\max(p_1), \dots, \max(p_L))[n] \quad (12)$$

Further, for token position n , for any particular token $v \in \{1, \dots, V\}$ that we could unmask in this position, we define the **vocabulary rank** of this choice v and denote it by j

$$j := \text{ArgSort}(p_n(1), \dots, p_n(V))[v] \quad (13)$$

So any candidate token that may be unmasked at sequence position can be denoted by c_{ij} in terms of its token position rank and its vocabulary rank. These indices (i, j) and the sorting method is visualized in Fig. 3. In this way, we define each of our draft blocks $\hat{X}_k^m(t_m)$ as consisting of the current block state $X_k(t)$ with a set of additionally unmasked tokens $\{c_{ij}\}$, such that $|\{c_{ij}\}| = \sum_{t < t' \leq t_m} S_{t'}$ as seen in Fig. 2. This ensures that the draft has the correct number of unmasked tokens for its timestep t_m . By leveraging the current target distribution in the construction of draft blocks, we avoid the need for an independent draft model.

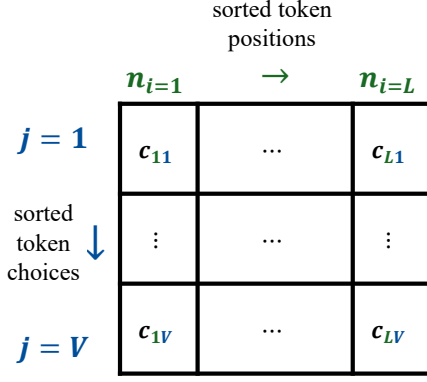


Figure 3: By leveraging the marginals of $p_{\text{TM}}(X(t))$ for each position in the block, we sort token candidates by i = token position rank, j = token choice rank.

Calibrating Draft Graphs Since we now have a source for draft blocks, the next step is to determine which candidate states are *most promising*. We know that it is to our advantage to structure these drafts as a directed graph, so we now need to configure the particular structure of this graph. Additionally, since there are L possible unmasking positions, each with V candidates to choose from, we desire that this construction not involve manual design.

To achieve this, we present a novel calibration algorithm to procedurally determine a fixed draft graph structure to be used throughout generation. This algorithm selects D **draft formulas**, each defined by a set $\{(i, j)\}$, allowing us to unmask the corresponding $\{c_{ij}\}$ for each draft block during the generation process. Intuitively, we wish to create drafts with assigned choices of (i, j) so that we may select their unmasked token contents once we have access to the current timestep’s probability distribution. Algorithm 2 details this procedure which involves running the vanilla dLLM on a calibration dataset and using the frequency of various sequences of (i, j) to create optimized directed draft graphs.

Figure 4 provides example of a directed draft graph that has been calibrated for LLaDA-Base-8B and HumanEval. As we can see, the resulting graph has a fairly intricate structure, with only a single level 1 node that leads to three level 2 nodes. This structure indicates that the token c_{11} occurred frequently as the immediate next token, with the second level of the tree attempting to explore combinations of c_{11} with potential second tokens, and so on for the subsequent levels.

We provide further intuition for this calibration strategy in Appendix C. We also discuss the costs of running this algorithm in Appendix C.2 and see that it requires < 50 calibration samples and < 30 minutes to perform offline on a single GPU. Additional ablations on the size of the calibration dataset are presented in Appendix D and ablations on alternative calibration strategies are included in Appendix E.

Thus, Spiffy’s method involves first running the calibration algorithm offline to determine an optimized draft graph structure with a number of drafts D . Speculation then be-

Algorithm 2: Draft Graph Structure Determination via Offline Calibration

Inputs: dLLM ϕ , sample *Dataset*, ‘look-ahead’ length for calibration $\mathcal{L}$, number of drafts budget D

Part 1: Collect calibration data
 $results \leftarrow []$
for sample $\in Dataset$ **do**
 $X \leftarrow \phi(sample)$ with:
denoising interval $[T, 0]$
recording of $p_{\text{TD}}(t) \quad \forall t$
Rewind the generation of X
for every timestep $t \in [T, 0]$ **do**
for $t' \in [t, t + \mathcal{L}]$ **do**
Unmasked tokens at current lookahead
 $x_1, \dots, x_k \leftarrow$ tokens unmasked at t'
Get (i, j) for each token using (12, 13)
Append $[(L = t' - t, \{(i, j)\})]$ to $results$
end for
end for
end for
$results = [(L = k, \{(i, j)\})]$ for $k \in 1, \dots, \mathcal{L}$
Part 2: All-possibilities-graph construction
for $k \in 1, \dots, \mathcal{L}$ **do**
Possibilities $\leftarrow$ sequences in $results$ with
 $L = 1, L = 2, \dots, L = k$
For each $s \in$ Possibilities, get $\bigcup_{e \in s} \{(i, j)\}$
Each s is represented by an unordered set $\{(i, j)\}$
 $w_{k1}, w_{k2}, w_{k3} \leftarrow$ top-3 most occurring sets $\{(i, j)\}$
 $count(w_{kv}) :=$ frequency of occurrence of w_{kv}
end for
 $G \leftarrow \{w_{kv}\}$ for $v \in 1, 2, 3, k \in 1, \dots, \mathcal{L}$
Each $w_{kv} = \{(i, j)\}$ is a draft formula
Part 3: Optimal subgraph selection
 $G' \leftarrow$ all subgraphs of G that satisfy (6, 7, 8) with size D
Find the optimal subgraph Q^*
 $Q^* = \arg \max_{Q \in G'} \left(count(q) + \sum_{p \in \text{parents}(q) \cap Q} count(p) \right)$
 $Q^* = \{w_{kv}\}$, a set of D draft formulas
Output Q^*

gins, using this graph configuration to determine the token candidates c_{ij} that will appear in each draft block. This draft graph is then verified in parallel to accept draft states, one per level. Speedup is determined by the total number of acceptances, and consequently, the number of denoising timesteps and model inferences saved.

5 Experiments

Following the setting of (Wu et al. 2025), we demonstrate the acceleration provided by Spiffy for LLaDA-Base-8B, LLaDA-Instruct-8B (Nie et al. 2025), and LLaDA-1.5-8B (Zhu et al. 2025) on popular downstream tasks, GSM8K (Cobbe et al. 2021), MATH (Hendrycks et al. 2021), MBPP (Austin et al. 2021b), and HumanEval (Chen et al. 2021), with generation length 256 and block size 32. We evalu-

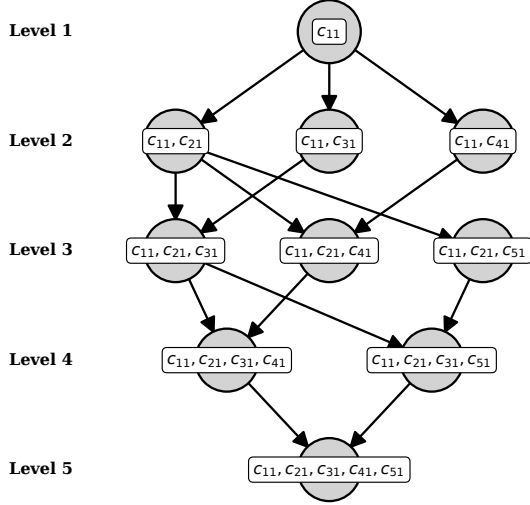


Figure 4: Directed draft graph calibrated using Alg. 2 for LLaDA-Base-8B with 50 samples from HumanEval.

ate on 350 samples each from GSM8K, MATH, and MBPP, and 100 samples from HumanEval using the standardized `lm-eval` library (Gao et al. 2024) with temperature 0.0 for a single run. We additionally compute the accuracy of the generations using the `exact_match` and `pass@1` metrics to validate the lossless property of our method. For each combination of dataset and model, we first run the offline calibration procedure to determine draft graph structures with $D = 3, 5, 8, 10$ nodes. Calibration is performed with 50 samples taken from the *training split* of each dataset to avoid contamination with the test set. Note: since HumanEval only provides a test set of 164 samples, we calibrate on the last 50 samples and evaluate only on the first 100 samples. Following other works, we calculate speedup in terms of savings in the Number of Function Evaluations (NFEs) of the dLLM.

$$\text{Speedup} := \frac{\text{Originally Required NFEs}}{\text{Final NFEs}} \quad (14)$$

We ensure that speedup is only calculated for blocks that occur before, or contain the first `<eos>` token in the output for each sample to avoid inflating our speedup results with tokens that may have been generated after the response.

5.1 Main Experiment

Table 1 displays the acceleration provided by Spiffy for LLaDA-8B-Base, LLaDA-8B-Instruct, and LLaDA-1.5 and we see that across tasks and models, Spiffy enables speedups of $2.80\text{--}3.04\times$. Notably, even if we allow only a smaller draft budget of $D = 3$ drafts, using the calibrated graph configuration allows us to reach speedups of up to $2.4\times$. The typical standard deviation in speedup for $D = 3, 5, 8, 10$ is $\pm 0.14, 0.19, 0.25, 0.26$ respectively.

Additionally, we see that model accuracy on downstream metrics is preserved up to minor differences in generations caused due to fully-bidirectional decoding of LLaDA as

noted in Sec 4.2 and precision errors. These precision discrepancies are attributed to `torch.bfloat16` GPU matrix multiplication accumulation errors for tensors of different sizes and are discussed further in Appendix A.2.

This result demonstrates that Spiffy achieves consistent speedups of up to $3\times$ across base, instruction-tuned, and preference-tuned models.

5.2 Multiplying Parallel Decoding Acceleration

While most open-source dLLMs set the default decoding rate to 1 token per model forward call, dLLMs may decode at arbitrary rates. For example, the LLaDA model family may decode 2 tokens per iteration by simply selecting the two highest-probability tokens per model forward call. Moreover, recent work such as `fast-dllm` (Wu et al. 2025) use a probability threshold to dynamically determine the number of tokens unmasked per iteration; the more conservative the decoding rate, the higher the accuracy. In our formulation for Spiffy, we assume arbitrary decoding rates to ensure that our method is compatible with a variety of parallel decoding schemes.

Table 2 applies Spiffy to two cases a) `hard-code`: where the number of tokens unmasked per iteration is hard-coded to 2 or 4 and b) `threshold` where the number of tokens unmasked is determined dynamically based on a probability threshold. In particular, all tokens that have probabilities of at least the threshold value will be unmasked. We present speedup numbers in the default setting where we calculate speedup by considering all the tokens generated in the requested 256 generation length, as well as in the case where we calculate speedup after excluding any tokens generated after the first `<eot>` token in the response, as we have done in Sec 5.1.

We demonstrate that by simply reusing the $D = 10$ draft graphs which were calibrated for the experiments in Sec. 5.1, Spiffy is able to *multiply* the benefits of other parallel decoding techniques while preserving their accuracy. This combined scheme is able to achieve speedups of up to $7.9\times$. Notably, Spiffy when combined with `threshold=0.9` is able to achieve a speedup of $5.18\times$ which is comparable to the speedup achieved by always decoding 4 tokens per iteration, but with significantly higher model accuracy. When we exclude tokens after the `<eot>` token from the speedup calculation, the parallel decoding speedup-contribution drops, but Spiffy is able to multiply the speedup of the system by a similar factor as before.

We expect that creating calibrated draft graphs specifically for the above parallel decoding speeds will lead to an additional boost in performance and defer this investigation to future work.

5.3 Analysis and Ablations

In this section, we highlight several results from our studies and provide further details for each in the corresponding appendices.

Losslessness We prove that our verification algorithm preserves the output distribution of the model and verify this

LLaDA-Base-8B					
Dataset	Number of draft blocks				
	0	3	5	8	10
GSM8K	1×	2.24×	2.50×	2.74×	2.80×
(exact_match)	0.71 ± 0.02	0.72 ± 0.03	0.68 ± 0.03	0.69 ± 0.03	0.68 ± 0.03
HumanEval	1×	2.40×	2.71×	3.00×	3.07×
(pass@1)	0.45 ± 0.05	0.42 ± 0.05	0.42 ± 0.05	0.45 ± 0.05	0.45 ± 0.05
MBPP	1×	2.39×	2.70×	2.97×	3.06×
(pass@1)	0.37 ± 0.03	0.36 ± 0.03	0.37 ± 0.03	0.38 ± 0.03	0.38 ± 0.03
MATH	1×	2.27×	2.54×	2.79×	2.88×
(exact_match)	0.25 ± 0.02	0.25 ± 0.02	0.23 ± 0.02	0.25 ± 0.02	0.23 ± 0.02
LLaDA-Instruct-8B					
Dataset	Number of draft blocks				
	0	3	5	8	10
GSM8K	1×	2.37×	2.68×	2.95×	3.04×
(exact_match)	0.56 ± 0.03	0.55 ± 0.03	0.54 ± 0.03	0.51 ± 0.03	0.53 ± 0.03
HumanEval	1×	2.29×	2.56×	2.80×	2.88×
(pass@1)	0.50 ± 0.05	0.52 ± 0.05	0.54 ± 0.05	0.54 ± 0.05	0.52 ± 0.05
MBPP	1×	2.30×	2.58×	2.84×	2.93×
(pass@1)	0.37 ± 0.03	0.38 ± 0.03	0.38 ± 0.03	0.37 ± 0.03	0.35 ± 0.03
MATH	1×	2.28×	2.57×	2.79×	2.89×
(exact_match)	0.24 ± 0.02	0.24 ± 0.02	0.22 ± 0.02	0.25 ± 0.02	0.24 ± 0.02
LLaDA-1.5-8B					
Dataset	Number of draft blocks				
	0	3	5	8	10
GSM8K	1×	2.40×	2.70×	2.96×	3.06×
(exact_match)	0.65 ± 0.03	0.62 ± 0.03	0.63 ± 0.03	0.68 ± 0.02	0.67 ± 0.03
HumanEval	1×	2.29×	2.57×	2.81×	2.90×
(pass@1)	0.56 ± 0.05	0.53 ± 0.05	0.53 ± 0.05	0.53 ± 0.05	0.52 ± 0.05
MBPP	1×	2.32×	2.60×	2.83×	2.93×
(pass@1)	0.37 ± 0.03	0.38 ± 0.03	0.39 ± 0.03	0.39 ± 0.03	0.38 ± 0.03
MATH	1×	2.30×	2.59×	2.82×	2.90×
(exact_match)	0.25 ± 0.02	0.26 ± 0.02	0.26 ± 0.02	0.29 ± 0.02	0.26 ± 0.02

Table 1: Spiffy achieves **speedups** of up to 3× for various dLLMs and tasks, while preserving **accuracy** on downstream metrics. Speedup := (Originally Required NFEs)/(Final NFEs).

experimentally, observing no change in accuracy on downstream metrics within the limits of numeric precision errors. This proof and verification are presented in Appendix A.

Overheads We perform detailed profiling of various aspects of Spiffy’s generation procedure and see that costs of enabling Spiffy, including drafting and verification, are negligible (0.2%–3.8%) as compared to the average model inference time. Further, we explore how model inference time scales with the input complexity in compute-bound systems and speculate that this scaling will be flatter in memory-bound systems. We include the above profiles in Appendix B. Additionally, in Appendix C.2, we discuss the resources required to run the graph calibration procedure and see that it takes < 30 minutes to perform on a single GPU.

Size of The Calibration Dataset We show that our algorithm successfully calibrates high-quality directed draft graphs with only 20–50 samples. This is studied in Appendix D, where we ablate on various sizes of the calibration dataset and evaluate the graph configurations that these settings produce. This result demonstrates that our calibration procedure is data-efficient, converging with a small number of samples.

Calibration Strategy We validate our choice of optimization metric in Part 3 of Alg. 2 and see that the draft graphs it produces consistently enable higher speedups than those determined by alternate criteria. We additionally provide visual examples of such calibrated graphs and describe this ablation in detail in Appendix E.

	Parallel Decoding Speedup	Parallel Decoding + Spiffy Speedup	Parallel Decoding Speedup (up to <eot>)	Parallel Decoding + Spiffy Speedup (up to <eot>)
default=1	1.00× 0.50 ± 0.05	2.95× 0.52 ± 0.05	1.00× 0.50 ± 0.05	2.88× 0.52 ± 0.05
hard-code=2	2.00× 0.46 ± 0.05	3.74× 0.43 ± 0.05	2.00× 0.46 ± 0.05	3.56× 0.43 ± 0.05
hard-code=4	4.00× 0.31 ± 0.05	5.28× 0.29 ± 0.05	4.00× 0.31 ± 0.05	4.97× 0.29 ± 0.05
threshold=0.9	3.32× 0.48 ± 0.05	5.18× 0.50 ± 0.05	2.65× 0.48 ± 0.05	4.40× 0.50 ± 0.05
threshold=0.8	4.69× 0.48 ± 0.05	6.84× 0.48 ± 0.05	3.53× 0.48 ± 0.05	5.15× 0.48 ± 0.05
threshold=0.7	6.43× 0.41 ± 0.05	7.88× 0.45 ± 0.05	4.55× 0.41 ± 0.05	5.83× 0.45 ± 0.05

Table 2: Spiffy multiplies the [speedup](#) achieved by other parallel decoding schemes resulting in speedups of upto 7.9× while preserving [accuracy](#). LLaDA-Instruct-8B is evaluated on HumanEval with the `pass@1` metric. `default=1` refers to decoding 1 token per iteration. `hard-code=x` refers to decoding a fixed number of tokens, x per iteration. `threshold=p` refers to unmasking all tokens that have probability at least p per iteration. Columns with (up to <eot>) calculate speedup after excluding tokens in blocks generated after the <eot> token. The speedup of the parallel decoding schemes is lower in this setting, but the multiplication provided by Spiffy remains consistent.

Calibration Domain Transfer We observe that in general, across settings, the calibrated graphs take on a stable core structure for a given number of nodes with only minor variations. Therefore, in a resource-conscious deployment, we may reuse calibrated graph configurations across multiple tasks without sacrificing performance. This is studied in Appendix F across datasets and models.

Per-block Acceptance Rates Spiffy is able to achieve consistent speedups across the each of the N individual blocks of the sequence, indicating that the calibrated draft graph generalizes well across blocks. Higher gains are also seen for the later blocks in the sequence, indicating that Spiffy’s speculations improve as it conditions on an increasing number of unmasked tokens. This study is included in Appendix G.

6 Discussion and Conclusion

Spiffy is a novel framework for lossless speculative decoding of diffusion LLMs that leverages draft blocks and their seamless verification to accelerate dLLM inference. We implement this framework via auto-speculation, enhance it by leveraging directed draft graphs, and further optimize these graphs through calibration. This method is shown to be effective for a variety of models, evaluated on multiple tasks, and combined with other parallel decoding schemes, achieving speedups of up to 7.9×. We analyze the overheads of our method, showing them to be negligible, and validate our method by ablating on various design choices.

As a framework, Spiffy’s utility also lies in its versatility. Our current auto-speculation implementation may be substituted with the use of an auxiliary draft model as smaller and more efficient diffusion language models become avail-

able. KV-caching may also be integrated by simply allowing the draft blocks to attend to the cached states. Spiffy’s formulation additionally generalizes to arbitrary denoising rates since we make no assumption about this schedule in our methods or algorithms. This ensures that our method remains a viable solution as frontier dLLMs require decreasingly fewer denoising steps to produce accurate outputs. Future investigations may explore the above ideas within Spiffy’s framework, allowing for powerful combinations of various techniques. Thus, with its losslessness and versatility, we propose Spiffy as a key component in unlocking high-speed dLLM inference.

References

- Arriola, M.; Gokaslan, A.; Chiu, J. T.; Yang, Z.; Qi, Z.; Han, J.; Sahoo, S. S.; and Kuleshov, V. 2025. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*.
- Austin, J.; Johnson, D. D.; Ho, J.; Tarlow, D.; and Van Den Berg, R. 2021a. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34: 17981–17993.
- Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; et al. 2021b. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Black Forest Labs; Batifol, S.; Blattmann, A.; Boesel, F.; Consul, S.; Diagne, C.; Dockhorn, T.; English, J.; English, Z.; Esser, P.; et al. 2025. FLUX. 1 Kontext: Flow Matching for In-Context Image Generation and Editing in Latent Space. *arXiv preprint arXiv:2506.15742*.
- Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901.
- Cai, T.; Li, Y.; Geng, Z.; Peng, H.; Lee, J. D.; Chen, D.; and Dao, T. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.
- Chen, C.; Borgeaud, S.; Irving, G.; Lespiau, J.-B.; Sifre, L.; and Jumper, J. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv:2302.01318*.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. D. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Cheng, S.; Bian, Y.; Liu, D.; Zhang, L.; Yao, Q.; Tian, Z.; Wang, W.; Guo, Q.; Chen, K.; Qi, B.; and Zhou, B. 2025. SDAR: A Synergistic Diffusion-AutoRegression Paradigm for Scalable Sequence Generation. *arXiv:2510.06303*.
- Christopher, J. K.; Bartoldson, B. R.; Ben-Nun, T.; Cardei, M.; Kailkhura, B.; and Fioretto, F. 2024. Speculative diffusion decoding: Accelerating language generation through diffusion. *arXiv preprint arXiv:2408.05636*.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Dao, T.; Fu, D.; Ermon, S.; Rudra, A.; and Ré, C. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35: 16344–16359.
- De Bortoli, V.; Galashov, A.; Gretton, A.; and Doucet, A. 2025. Accelerated diffusion models via speculative sampling. *arXiv preprint arXiv:2501.05370*.
- Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 4171–4186.
- Du, Y.; Li, S.; and Mordatch, I. 2020. Compositional visual generation with energy based models. *Advances in Neural Information Processing Systems*, 33: 6637–6647.
- Esser, P.; Kulal, S.; Blattmann, A.; Entezari, R.; Müller, J.; Saini, H.; Levi, Y.; Lorenz, D.; Sauer, A.; Boesel, F.; Podell, D.; Dockhorn, T.; English, Z.; and Rombach, R. 2024. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. In *Forty-first International Conference on Machine Learning*.
- Fu, Y.; Bailis, P.; Stoica, I.; and Zhang, H. 2024a. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*.
- Fu, Y.; Bailis, P.; Stoica, I.; and Zhang, H. 2024b. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*.
- Gao, L.; Tow, J.; Abbasi, B.; Biderman, S.; Black, S.; DiPofi, A.; Foster, C.; Golding, L.; Hsu, J.; Le Noac’h, A.; Li, H.; McDonell, K.; Muennighoff, N.; Ociepa, C.; Phang, J.; Reynolds, L.; Schoelkopf, H.; Skowron, A.; Sutawika, L.; Tang, E.; Thite, A.; Wang, B.; Wang, K.; and Zou, A. 2024. The Language Model Evaluation Harness.
- Goel, R.; Gagrani, M.; Jeon, W.; Park, J.; Lee, M.; and Lott, C. 2024. Direct alignment of draft model for speculative decoding with chat-fine-tuned llms. *arXiv preprint arXiv:2403.00858*.
- Google. 2025. Gemini Diffusion. Accessed: 2025-07-25.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv:2103.03874*.
- Ho, J.; Jain, A.; and Abbeel, P. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33: 6840–6851.
- Hong, F.; Yu, G.; Ye, Y.; Huang, H.; Zheng, H.; Zhang, Y.; Wang, Y.; and Yao, J. 2025. Wide-In, Narrow-Out: Revokable Decoding for Efficient and Effective DLLMs. *arXiv:2507.18578*.
- Hu, H.; Das, A.; Sadigh, D.; and Anari, N. 2025. Diffusion Models are Secretly Exchangeable: Parallelizing DDPMs via Autospeculation. *arXiv preprint arXiv:2505.03983*.
- Inception AI Labs; Khanna, S.; Kharbanda, S.; Li, S.; Varma, H.; Wang, E.; Birnbaum, S.; Luo, Z.; Miraoui, Y.; Palrecha, A.; Ermon, S.; Grover, A.; and Kuleshov, V. 2025. Mercury: Ultra-Fast Language Models Based on Diffusion. *arXiv:2506.17298*.
- Israel, D.; Broeck, G. V. d.; and Grover, A. 2025. Accelerating Diffusion LLMs via Adaptive Parallel Decoding. *arXiv preprint arXiv:2506.00413*.
- Jeon, W.; Gagrani, M.; Goel, R.; Park, J.; Lee, M.; and Lott, C. 2024. Recursive speculative decoding: Accelerating llm

- inference via sampling without replacement. *arXiv preprint arXiv:2402.14160*.
- Leviathan, Y.; Kalman, M.; and Matias, Y. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 19274–19286. PMLR.
- Li, S.; Kallidromitis, K.; Bansal, H.; Gokul, A.; Kato, Y.; Kozuka, K.; Kuen, J.; Lin, Z.; Chang, K.-W.; and Grover, A. 2025. Lavidia: A large diffusion language model for multi-modal understanding. *arXiv preprint arXiv:2505.16839*.
- Li, Y.; Wei, F.; Zhang, C.; and Zhang, H. 2024. Eagle-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*.
- Lin, F.; Yi, H.; Yang, Y.; Li, H.; Yu, X.; Lu, G.; and Xiao, R. 2025. BiTA: Bi-directional tuning for lossless acceleration in large language models. *Expert Systems with Applications*, 279: 127305.
- Liu, Z.; Yang, Y.; Zhang, Y.; Chen, J.; Zou, C.; Wei, Q.; Wang, S.; and Zhang, L. 2025. dllm-cache: Accelerating diffusion large language models with adaptive caching. *arXiv preprint arXiv:2506.06295*.
- Lou, A.; Meng, C.; and Ermon, S. 2023. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*.
- Luxembourg, O.; Permuter, H.; and Nachmani, E. 2025. Plan for Speed–Dilated Scheduling for Masked Diffusion Language Models. *arXiv preprint arXiv:2506.19037*.
- Ma, X.; Yu, R.; Fang, G.; and Wang, X. 2025. dkv-cache: The cache for diffusion language models. *arXiv preprint arXiv:2505.15781*.
- Miao, X.; Oliaro, G.; Zhang, Z.; Cheng, X.; Wang, Z.; Zhang, Z.; Wong, R. Y. Y.; Zhu, A.; Yang, L.; Shi, X.; et al. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 932–949.
- Nie, S.; Zhu, F.; You, Z.; Zhang, X.; Ou, J.; Hu, J.; Zhou, J.; Lin, Y.; Wen, J.-R.; and Li, C. 2025. Large Language Diffusion Models. *arXiv:2502.09992*.
- Saharia, C.; Chan, W.; Saxena, S.; Li, L.; Whang, J.; Denton, E.; Ghasemipour, S. K. S.; Gontijo-Lopes, R.; Ayan, B. K.; Salimans, T.; Ho, J.; Fleet, D. J.; and Norouzi, M. 2022. Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding. In Oh, A. H.; Agarwal, A.; Belgrave, D.; and Cho, K., eds., *Advances in Neural Information Processing Systems*.
- Sahoo, S.; Arriola, M.; Schiff, Y.; Gokaslan, A.; Marroquin, E.; Chiu, J.; Rush, A.; and Kuleshov, V. 2024. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37: 130136–130184.
- Shaul, N.; Gat, I.; Havasi, M.; Severo, D.; Sriram, A.; Holderrieth, P.; Karrer, B.; Lipman, Y.; and Chen, R. T. 2024. Flow matching with general discrete paths: A kinetic-optimal perspective. *arXiv preprint arXiv:2412.03487*.
- Shi, J.; Han, K.; Wang, Z.; Doucet, A.; and Titsias, M. 2024. Simplified and generalized masked diffusion for discrete data. *Advances in neural information processing systems*, 37: 103131–103167.
- Singhal, R.; Horvitz, Z.; Teehan, R.; Ren, M.; Yu, Z.; McKeeown, K.; and Ranganath, R. 2025. A general framework for inference-time scaling and steering of diffusion models. *arXiv preprint arXiv:2501.06848*.
- Sohl-Dickstein, J.; Weiss, E.; Maheswaranathan, N.; and Ganguli, S. 2015. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. In Bach, F.; and Blei, D., eds., *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, 2256–2265. Lille, France: PMLR.
- Song, Y.; Sohl-Dickstein, J.; Kingma, D. P.; Kumar, A.; Ermon, S.; and Poole, B. 2020. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*.
- Wang, Z.; Zhang, R.; Ding, K.; Yang, Q.; Li, F.; and Xiang, S. 2024. Continuous speculative decoding for autoregressive image generation. *arXiv preprint arXiv:2411.11925*.
- Wu, C.; Zhang, H.; Xue, S.; Liu, Z.; Diao, S.; Zhu, L.; Luo, P.; Han, S.; and Xie, E. 2025. Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding. *arXiv:2505.22618*.
- Ye, J.; Xie, Z.; Zheng, L.; Gao, J.; Wu, Z.; Jiang, X.; Li, Z.; and Kong, L. 2025. Dream 7B.
- Zhang, J.; Wang, J.; Li, H.; Shou, L.; Chen, K.; Chen, G.; and Mehrotra, S. 2023. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*.
- Zhu, F.; Wang, R.; Nie, S.; Zhang, X.; Wu, C.; Hu, J.; Zhou, J.; Chen, J.; Lin, Y.; Wen, J.-R.; et al. 2025. LLaDA 1.5: Variance-Reduced Preference Optimization for Large Language Diffusion Models. *arXiv preprint arXiv:2505.19223*.

A Proof of Losslessness

Let the current state of the sequence be $X(t_0) = X'; X_k(t_0)$. We wish to show that when the denoising progresses from $t_0 \rightarrow t_0 - 2$, we achieve the same result whether we had used only the true (target) probabilities or accepted a draft block using Spiffy and skipped to timestep $t_0 - 2$.

First, define the process of sampling the next state of a block X from some probability distribution p as a function $\text{Next}(X, p)$. This is simply the process of using p to performing ranking and picking of token positions to be unmasked in X described in Steps 1-4 of Alg. 1 and is common to both speculative and non-speculative implementations.

A.1 Proof

Vanilla decoding output: Let $X_k^*(t_0 - 2)$ be the final state of the sequence with no speculation. From $t_0 \rightarrow t_0 - 1$, we may obtain the next state as:

$$\begin{aligned} p_{\text{TD}}(t_0 - 1) &\leftarrow \\ p_{\theta}(X_k(t_0 - 1) \mid X'; X_k(t_0)) &\end{aligned} \quad (15)$$

$$\begin{aligned} X_k(t_0 - 1) &\leftarrow \\ \text{Next}(X_k(t_0), p_{\text{TD}}(t_0 - 1)) &\end{aligned} \quad (16)$$

This state becomes the true state of the block and inference is performed on it once more by the dLLM to advance to $t_0 - 2$

$$\begin{aligned} p_{\text{TD}}(t_0 - 2) &\leftarrow \\ p_{\theta}(X_k(t_0 - 2) \mid X'; X_k(t_0 - 1)) &\end{aligned} \quad (17)$$

$$\begin{aligned} X_k^*(t_0 - 2) &\leftarrow \\ \text{Next}(X_k(t_0 - 1), p_{\text{TD}}(t_0 - 2)) &\end{aligned} \quad (18)$$

Spiffy output: Let $X_k^+(t_0 - 2)$ be the state of the sequence if we first accept a draft block $\hat{X}_k^m$ at timestep t_m using Alg. 1, then treat it as the true block, and then continue to the next iteration of the algorithm where we perform unmasking once more.

First, in the case of speculation, we also sample $X_k(t_0 - 1)$ as usual in Steps 1-4 as is done in (16). The next step of the algorithm is to check for acceptance or rejection.

Case 1 (rejection): For any of the drafts $m \in \{1, \dots, D\}$, if $\hat{X}_k^m$ is rejected, the algorithm continues to the next available draft and checks again for acceptance or rejection. If all drafts are rejected, we simply break and output $X_k(t_0 - 1)$ as the true block state. At this point, in the next iteration, we will call the dLLM once more and this state $X_k(t_0 - 1)$ will be used to compute p_{TD} as is done in (17). Thus, we obtain $X_k^+(t_0 - 2)$ in the same way and from the same distribution as $X_k^*(t_0 - 2)$ and thus they are equal.

Case 2 (acceptance): If $\hat{X}_k^m$ is accepted. We now set $p_{\text{TD}} \leftarrow p_{\text{DD}}^m(t_m - 1)$ and restart the algorithm. In this next step, we sample from this distribution to obtain

$$\begin{aligned} X_k^+(t_0 - 2) &\leftarrow \\ \text{Next}(\hat{X}_k^m(t_m - 1), p_{\text{DD}}^m(t_m)) &\end{aligned} \quad (19)$$

Since we previously had acceptance, we first have that $t_m == t_0 - 1$. Moreover, we have that $\hat{X}_k^m == X_k(t_0 - 1)$ from the acceptance condition. So we see that:

$$p_{\text{DD}}^m(t_m - 1) = p_{\text{DD}}^m(t_0 - 2)$$

and, since by definition

$$p_{\text{DD}}^m(t_m - 1) = p_{\theta}(X_k(t_m - 1) \mid X'; \hat{X}_k^m)$$

we have that

$$p_{\text{DD}}^m(t_0 - 2) = p_{\theta}(X_k(t_0 - 2) \mid X'; X_k(t_0 - 1)) \quad (20)$$

$$= p_{\text{TD}}(t_0 - 2) \quad (21)$$

So, in reality, we sample

$$\begin{aligned} p_{\text{DD}}^m(t_0 - 2) &\leftarrow \\ p_{\theta}(X_k(t_0 - 2) \mid X'; X_k(t_0 - 1)) & \\ = p_{\text{TD}}(t_0 - 2) &\end{aligned} \quad (22)$$

$$\begin{aligned} X_k^+(t_0 - 2) &\leftarrow \\ \text{Next}(X_k^m(t_0 - 2), p_{\text{DD}}^m(t_0 - 2)) & \\ = \text{Next}(X_k^m(t_0 - 2), p_{\text{TD}}(t_0 - 2)) &\end{aligned} \quad (23)$$

which is the same as obtaining $X_k^*(t_0 - 2)$ in (17), (18) $\square$

A.2 Experimental Verification

We note that achieving the losslessness described above requires generating draft probabilities such that $p_{\text{DD}}^m(t_0 - 2) = p_{\text{TD}}(t_0 - 2)$. That is, the output distribution of the draft block should be identical to the distribution produced by the same block during true vanilla inference. In some settings, ensuring this is computationally expensive because the draft block must attend to the same information as the true block, but must also attend to the original block in order to generate the target distribution. Concretely, suppose we have 3 blocks X_1, X_2, X_3 and we are currently decoding $X_2(t)$ with a draft block $X_2^*(t - 1)$. Since X_2^* attends to the entire sequence (excluding X_2), but X_1 and X_3 in turn attend to X_2 , the draft block is indirectly attending to the current block at a previous timestep t . This leakage of information would lead to drift in the output hidden states of the draft block across layers. However, two factors mitigate this deviation. First, X_1 has already been fully decoded in semi-autoregressive inference, so its hidden states remain stable during decoding (Wu et al. 2025). Second, since X_3 consists entirely of masked tokens, both X_2 and X_2^* place a low attention mass on this block. Due to these factors, the shift in output distribution is minimal.

We validate our claim above by computing the downstream accuracy of the LLaDA model with full-sequence bidirectional decoding with and without Spiffy and comparing their resulting metric scores. These results are displayed in Table 1. Each task defined by lm-eval has an associated metric which is computed automatically along with an associated standard error. Table 1 demonstrates that even in the case of fully-bidirectional decoding, Spiffy is indeed a lossless method with respect to the vanilla model inference. In

this framework, we may also conclude that when a dLLM is run with blockwise causal masking (Cheng et al. 2025), the previous hidden states do not attend to future hidden states, which will ensure that the draft and true distribution remain identical. Similarly, if a dLLM is run with dual KV caches, before and after the current block (Wu et al. 2025), the keys and values of X_1 and X_3 remain constant, resulting in an exact draft distribution.

We further investigate minor differences in the metric values, on the order 0.01, and attribute them to minor variations in generated outputs due to precision loss caused by `torch.bfloat16` matrix multiplication on CUDA GPUs. This is caused due to differing accumulation orders for tensors of differing sizes. In particular, even though each of the 10 draft blocks in Spiffy with $D = 10$ are independent, matrix multiplications with this extended tensor will have a slightly different accumulation order as compared to an input tensor with only $D = 3$ draft blocks added to it. This leads to minor differences in generations and due to cumulative nature of errors, may lead to different final results for tasks that compute `exact_match` or `pass@1` as their metric. For applications where precision is critical, we recommend padding the input tensor to 4096 to ensure predictable accumulation on CUDA GPU’s in reduced precision. We do not adopt this method here as it is prohibitively expensive and minor differences do not affect the output significantly as demonstrated by Table 1. The PyTorch documentation on numerical accuracy may be referred to for further explanation of floating point error accumulation in such settings.

B Overheads of Spiffy

In this section, we examine the overheads of various stages of Spiffy including token position ranking, vocabulary sorting, drafting, verification, custom attention mask construction, and custom position id construction and normalize them to a % of the average model inference time for ease of comparison. We collect this data while performing the experiments displayed in Table 1 and consider # drafts $D = 3, 5, 8, 10$. All profiling was done on a single 80GB NVIDIA A100 GPU with `torch==2.2.1` `transformers==4.45.0`, with vectorized PyTorch operations. Since Spiffy requires a custom attention mask that is not yet supported in Flash-Attention (Dao et al. 2022), we disable flash attention for all our experiments and use a naïve vectorized implementation of attention for a fair comparison to non-speculation cases. We additionally restrict our analysis to the statistics collected for GSM8K only, since varying lengths of inputs prompts across datasets would impact the average model inference time.

We see in Table 3 that the costs of the various stages of generation with Spiffy are negligible as compared to the model inference time, with drafting, custom attention mask construction, and custom position id construction each taking $< 0.5\%$ of the model inference time. Verification takes $< 3\%$ even with $D = 10$. Vocabulary sorting, which is performed to gather the top-k token choices in the vocabulary in order to create draft blocks, takes $\sim +2\%$ of the model inference time compared to vanilla inference which needs

to determine only the top-1 choice. This analysis additionally reveals that the time taken for token position ranking, necessary for both vanilla inference and Spiffy, is disproportionately high - future optimizations made to this step would be beneficial to both Spiffy and vanilla inference. This analysis thus shows that even a simple implementation of Spiffy involves little-to-no overheads and may further be optimized in latency-critical production scenarios by employing multi-threading and other techniques.

Overheads as % of model call time					
Number of draft blocks					
	0	3	5	8	10
Model	100.0	100.0	100.0	100.0	100.0
Vocab sort	1.6	3.8	3.8	3.6	3.3
Pos sort	188.0	179.0	181.0	179.0	177.0
Drafting	0.0	0.3	0.4	0.5	0.5
Mask	0.0	0.2	0.2	0.2	0.3
Pos ids	0.0	0.2	0.2	0.2	0.3
Verify	0.0	1.8	2.0	2.4	2.3

Table 3: Overheads of Spiffy, normalized to a % of the model inference time for # draft blocks $D = 0, 3, 5, 8, 10$. $D = 0$ corresponds to vanilla inference. The overheads of enabling Spiffy are negligible, each $< 5\%$ of the model inference time.

We additionally examine the cost of inference of the model as it scales with the # drafts $D = 0, 3, 5, 8, 10$ in Fig. 5. We see that the model inference time increases approximately linearly as we increase the number of draft blocks from $D = 0 \rightarrow 10$ with this simple implementation. We expect that with more efficient, custom attention kernels, inference time will be increasingly memory-bound and this curve will be significantly flatter, further boosting Spiffy’s performance.

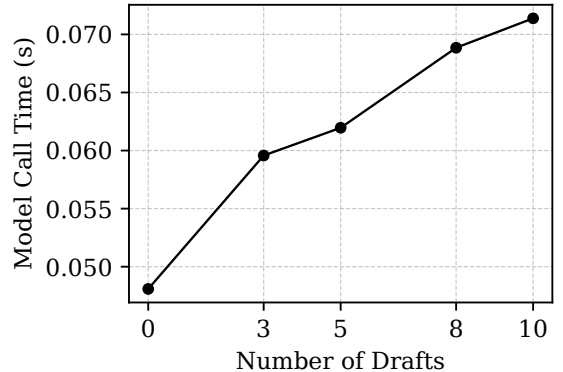


Figure 5: The model inference time sees a gradual increase from $D = 0$ to $D = 10$ due to inefficient attention computation. This curve is likely to be flatter in memory-bound systems where latency is less dependent on the input complexity.

C Calibration of Directed Draft Graphs

C.1 Intuition

In Alg. 2 we optimize the following expression :

$$Q^* = \arg \max_{Q \in G'} \sum_{q \in Q} \left(\text{count}(q) + \sum_{p \in \text{parents}(q) \cap Q} \text{count}(p) \right) \quad (24)$$

Each $q \in Q$ represents a sequence of tokens that has appeared in order. For example, (c_{11}, c_{21}, c_{31}) . $\text{count}(q)$ thus represents how often this particular sequence has occurred in the collected calibration data.

We have an additional constraint that a particular sequence may only occur if at least one of its parent sequences occur. In the above example, (c_{11}, c_{12}) or (c_{11}, c_{31}) or (c_{21}, c_{31}) must first occur for sequence of length 3 to occur. Moreover, since we have the unique advantage of each node potentially having multiple parent nodes (as in the example above), we would like to incorporate that information into our optimization. In particular, a node is more likely to have occurred if its parents are likely to occur. Thus, the total score of a node sums its frequency $\text{count}(q)$ with the sum of its parents' frequencies $\sum_p \text{count}(p)$.

C.2 Time to Perform Calibration

The cost of collecting calibration data is a multiple of the number of calibration samples used and the time to process the sample with the dLLM. In our experiments calibrating LLaDA-8B-Base for 50 samples, the time to process a single sample is ~ 40 seconds for a total calibration time of ~ 30 minutes on a single NVIDIA A100 GPU. The time taken to perform the optimization step in (24) using a brute-force breadth-first-search approach is ~ 5 minutes. Note, in Appendix D we ablate on the number of calibration samples and see that we may achieve comparable performance with only 20 or even 10 calibration samples which would reduce the total amount of time required for calibration by a corresponding factor. In sum, calibration is an inexpensive procedure that can lead to high-quality draft graph structures.

D Ablation on The Number of Calibration Samples Used

In this section, we provide an ablation on the **size** of the calibration dataset. Simply, this is the number of samples we pass through the vanilla dLLM to collect our calibration dataset. Note, in all cases, samples are only drawn from the train dataset to avoid contamination with the test set during speedup evaluation. In particular, 10, 20, 50 samples are drawn from the train set of GSM8K and calibration data is collected using LLaDA-8B-Base. Directed draft graphs are then generated using Alg. 2 for number of drafts budgets $D = 3, 5, 8, 10$. Evaluation is then performed using this graph as the configuration for Spiffy, with 200 samples of GSM8K to profile the speedup achieved for each setting. We see in Fig. 6 that the calibration procedure is data-efficient and the speedup achieved by the resulting graphs is very

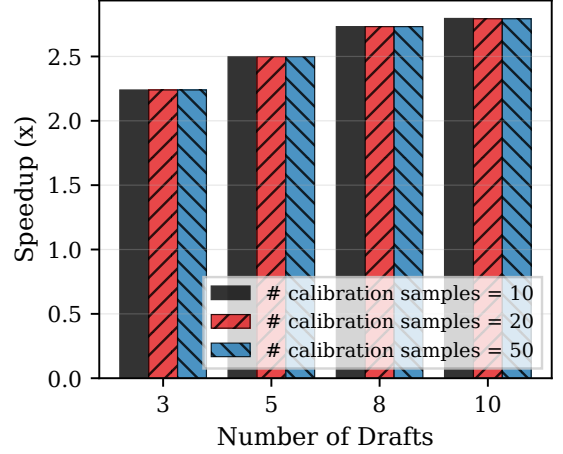


Figure 6: Speedup for varying numbers of calibration samples with LLaDA-Base-8B, with 10, 20, 50 samples from GSM8K, across # draft blocks $D = 3, 5, 8, 10$. All 3 cases show similar speedup, demonstrating that the calibration procedure is data efficient.

similar across # drafts budget $D = 3, 5, 8, 10$, regardless of whether we use 10, 20, or 50 samples.

Indeed, inspecting the graphs for one of these cases with $D = 10$, we see that the graphs that were calibrated using 20 and 50 samples are the same while the graph that used 10 samples is slightly different from level 4 onwards. Both these variants achieve comparable speedup, thus showing that multiple high-acceptance-rate directed draft graph structures may exist and may be discovered via our calibration algorithm. In our experiments, we set the number of samples to 50 for consistency, but it is likely that 20 or even 10 calibration samples would suffice and reduce the total time needed for calibration.

E Ablation on The Calibration Strategy Used

Thus far, we have been considering the optimization strategy in (25) to perform calibration. Let us refer to it as **degree-1-accumulation**:

$$Q^* = \arg \max_{Q \in G'} \sum_{q \in Q} \left(\text{count}(q) + \sum_{p \in \text{parents}(q) \cap Q} \text{count}(p) \right) \quad (25)$$

In this section, we consider various alternate draft graph construction strategies that are similar to degree-1-accumulation, yet we will see, produce different draft structures and speedup times.

We may define **degree-0-accumulation** as simply :

$$Q^* = \arg \max_{Q \in G'} \sum_{q \in Q} \text{count}(q) \quad (26)$$

We may also consider **total-accumulation** where we use a recursive metric, adding the total count of each parent node

to the count of the child node:

$$totalcount(q) = count(q) + \sum_{p \in parents(q) \cap Q} totalcount(p) \quad (27)$$

$$Q^* = \arg \max_{Q \in G'} \sum_{q \in Q} totalcount(q) \quad (28)$$

These three strategies are natural extensions of each other, with increasing complexity. We study their performance by allowing them to each produce optimized graphs of $D = 3, 5, 8, 10$ using 50 calibration samples collected from the train set of LLaDA-Base-8B on GSM8K. We then profile their speedup using 200 test samples of GSM8K. The results of these experiments are provided in Table 4. We see that for all values of # draft blocks, $D = 3, 5, 8, 10$, degree-1-accumulation results in draft graphs that have consistently higher speedups than degree-0-accumulation and total-accumulation. This indicates that while degree-0-accumulation is a simpler strategy, information about the parent nodes is useful to incorporate. On the other hand, though total-accumulation sets the score of a node using the scores of all its ancestors, this may lead to incorporation of information from unrelated nodes. degree-1-accumulation seems to be an ideal balance of the two strategies.

LLaDA-Base-8B, GSM8K				
	Number of draft blocks			
Draft Strategy	3	5	8	10
degree-0-acc	2.24×	2.45×	2.56×	2.56×
degree-1-acc	2.24×	2.50×	2.73×	2.80×
total-acc	2.24×	2.45×	2.62×	2.75×

Table 4: Ablation on the choice of draft calibration strategy. The draft graphs produced by degree-1-accumulation consistently result in a higher inference-time speedup than those produced by the other strategies. This holds across choices of number of draft blocks.

We may also visualize the $D = 8$ case for each of the strategies in Fig. 7. We see that in fact, degree-0-accumulation reverts to picking a single straight chain consisting of $c_{11}, c_{21}, \dots, c_{81}$ due to the high frequency of this particular sequence occurring. degree-1-accumulation and total-accumulation result in more intricate structures, with degree-1-accumulation prioritizing adding extra nodes at lower levels. The better performance of degree-1-accumulation indicates that it is important to have a wider graph initially to ensure that the baseline acceptance is high.

F Analysis of Calibrated Graph Structures Across Datasets and Models

We wish to explore whether the optimized calibration graphs show significant differences for various settings of models and datasets. To this end, we optimize a set of calibrated graphs for LLaDA-Base-8B, LLaDA-Instruct-8B, and LLaDA-1.5-8B for each of GSM8K, HumanEval,

MBPP, and MATH, with a fixed budget of 50 samples per setting. We observe that across models and tasks, the calibrated procedure converges to draft graphs that remain stable for each budget of # drafts, exhibiting only minor variations. A variation that may be noted is the calibrated graph for LLaDA-Base-8B on the MATH dataset with $D = 10$ nodes has a depth of 6 as compared to other graphs with depth 5. Regardless, with only minor variations, this calibration procedure is thus seen to produce draft graphs that generalize across tasks. This is encouraging as it allows for a single draft graph to be calibrated once and then deployed effectively in a variety of settings.

An example of this stability can be seen by comparing graphs in Fig. 4 for a $D = 10$ graph calibrated on HumanEval with Fig. 7 (c) for a $D = 10$ graph calibrated on GSM8K.

G Analysis of Acceptance Rates for Each Block

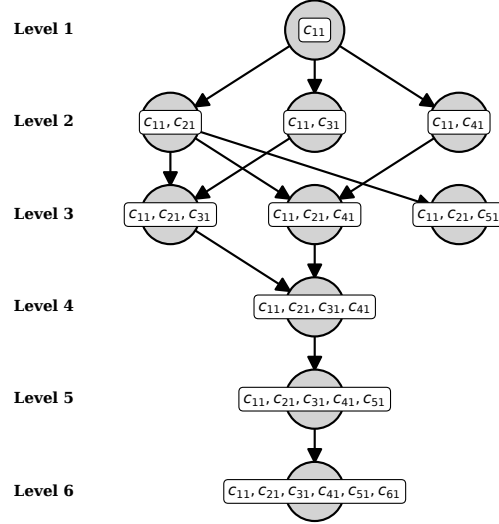
In this section, we study the individual speedup statistics of each of the decoding blocks. Recall, each block is of length L and we have a total of N blocks. The objective is to determine whether any particular block(s) experience(s) greater or lesser speedup than the others.

We analyze the data collected while generating Table 1 and in particular, examine the average statistics of LLaDA-8B-Base across GSM8K, HumanEval, MBPP, and MATH for # drafts budgets $D = 3, 5, 8, 10$. For each dataset, we consider the average speedup statistics of all the blocks that compose the sequence up to the average generation length. That is, up to the number of tokens produced before the end of text token. This allows our analysis to be fair and not consider extraneous tokens generated after the response has completed. With this method, we consider the first 7 blocks of size 32 for GSM8K and HumanEval and all 8 blocks for the other two datasets.

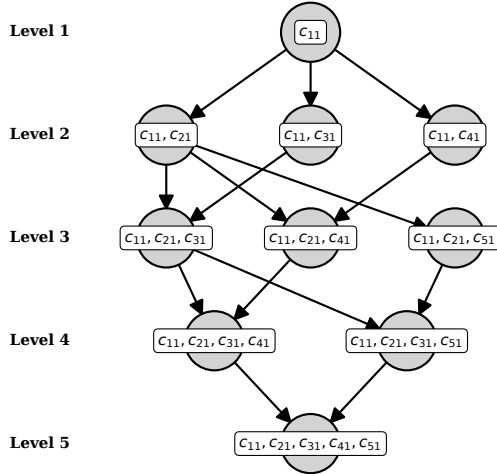
The results of this study are displayed in Fig. 9. We see that MBPP and MATH, with longer output sequences, demonstrate similar trends where Spiffy consistently achieves higher speedups for later blocks in the sequence. This seems to indicate that conditioning on more data allows Spiffy to better predict future draft block states, an intuitive result. When considering HumanEval, we see similar speedup across various blocks. For GSM8K, we see a spike around block # 2 and decreasing average speedup for the remaining blocks. We attribute this to the shorter responses for the model on this dataset, biasing the average per-block statistics for later blocks for this dataset.

Our analysis demonstrates that the directed draft graphs used by Spiffy are able to generalize well across the various blocks in the sequence. Spiffy is also able to produce higher speedups when conditioned on more of the sequence. At the same time, this study opens up the opportunity to further optimize these directed graphs. In particular, instead of creating a single draft graph to be used while speculating all blocks, we may consider $K \leq N$ groups of blocks and calibrate K unique graphs, one for each group. Each graph may be calibrated using Alg. 2 by only considering the statistics

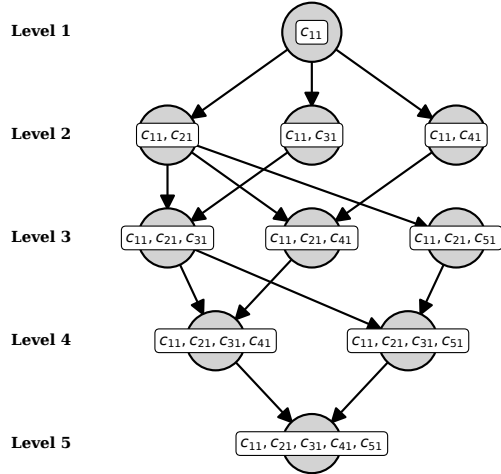
of that group of blocks. This may further increase the efficacy of these directed graphs when speculating tokens in these blocks. We suggest this as a potential direction of investigation for future works.



(a) 10 calibration samples



(b) 20 calibration samples



(c) 50 calibration samples

Figure 7: Effect of number of calibration samples on graph structure for # drafts budget $D = 10$. Graphs calibrated with 10 samples favor long chains of drafts while 20 and 50 sample calibration arrive at the same configuration, indicating data-efficient calibration convergence. Calibration was performed with LLaDA-Base on GSM8K.

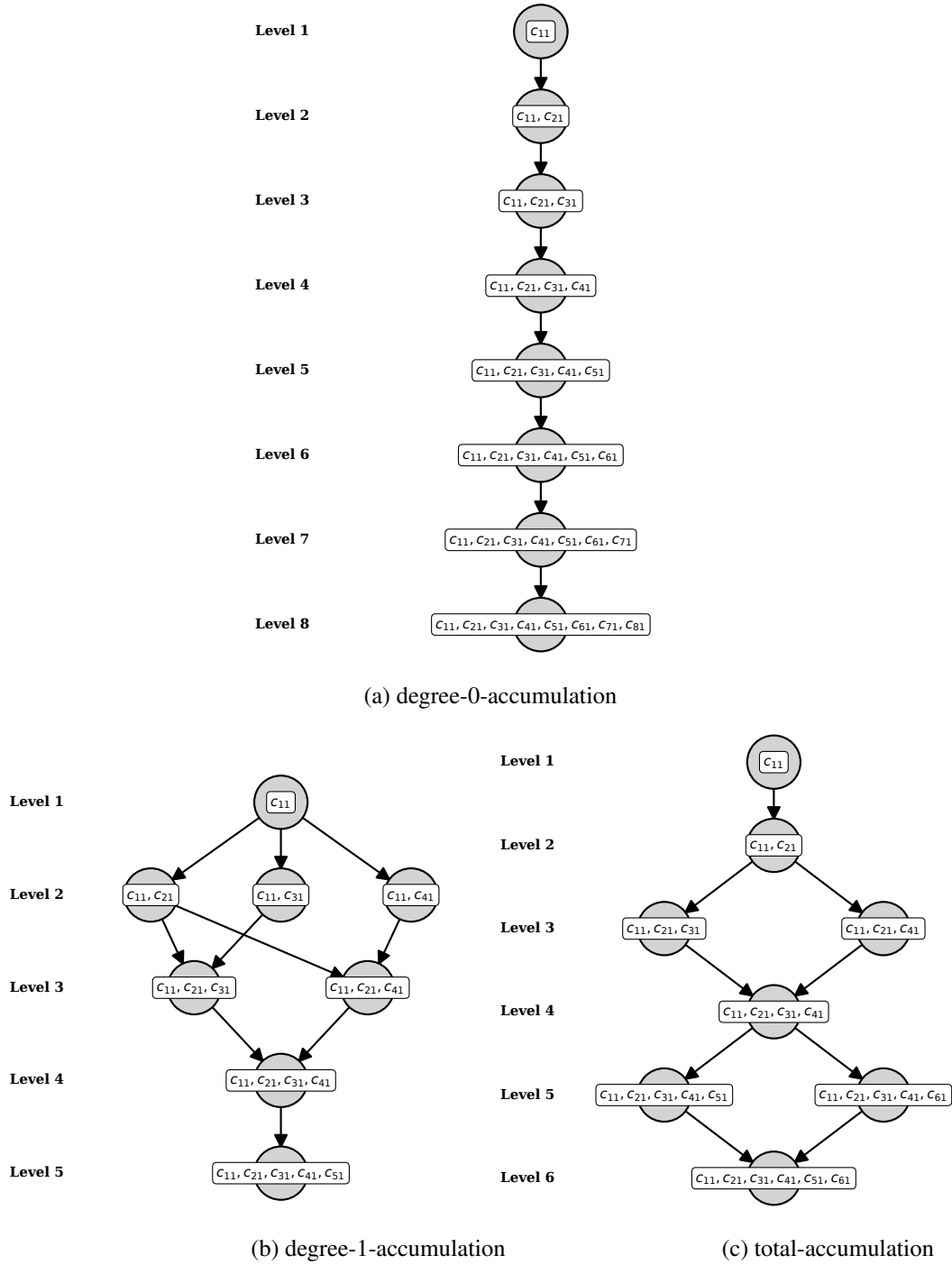


Figure 8: Effect of calibration strategy on graph structure for drafts budget $D = 8$. Degree-1-accumulation results in the highest acceptance rates due to it prioritizing more nodes at lower levels as compared to the other strategies which result in graphs with increased depth.

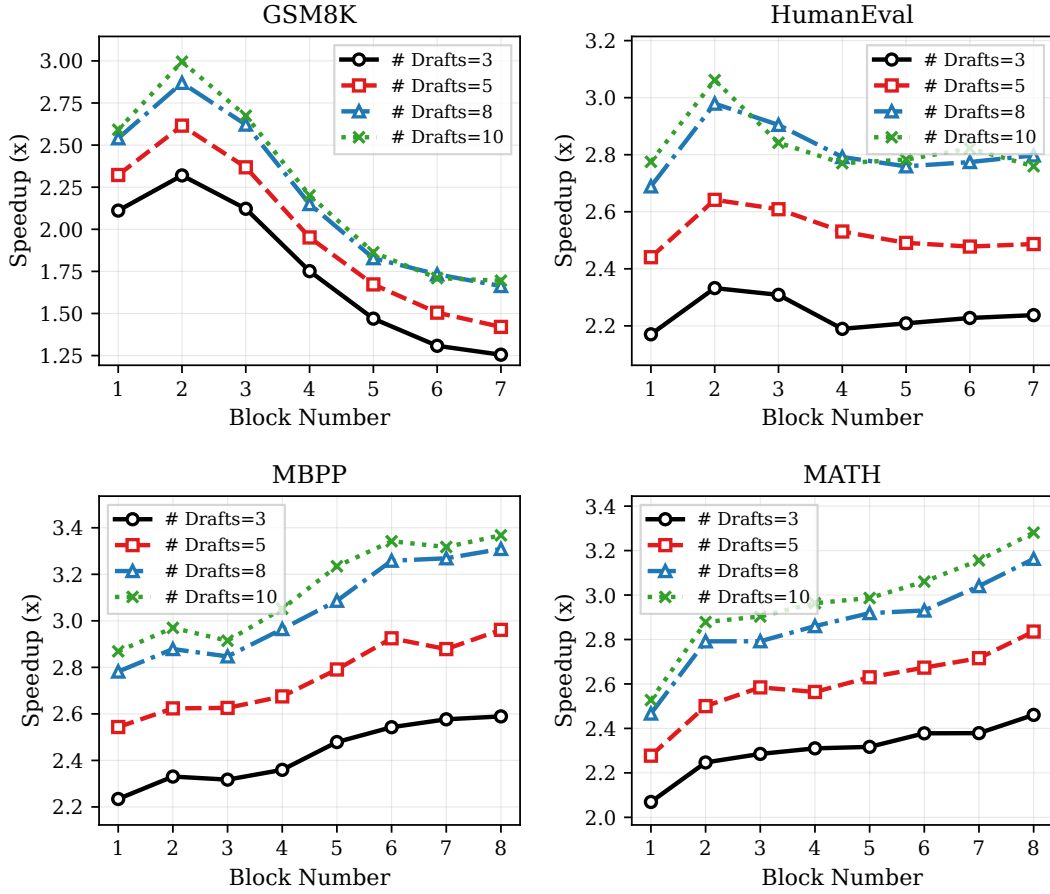


Figure 9: Speedup across each denoising block of size 32 for GSM8K, HumanEval, MBPP, MATH, with # drafts budgets $D = 3, 5, 8, 10$. Speedup is consistent across blocks, with MBPP and MATH showing an uptick in speedup for later blocks due to better conditioning. The average speedup for GSM8K, with shorter responses, is lower for later blocks since only count blocks that occur before the `<eos>` token. Speedup is consistently higher when using a higher # of drafts.